

.NET programming interview assignment

Hello.

Please do as much of this assignment as you are able to complete **within 3 hours**. You can choose to make use of any third-party technology or library for .NET C#.

Good luck with the processing. :)

Task

Your task is to implement a RESTful back-end API for registering and retrieving shop articles. Data may not be persistent between runs and you can use a data structure list or in-mem database for testing and debugging. Implement the following HTTP end-points according to the corresponding contracts:

The article object

Property	Required?	Format
article_id	optional	Number > 0. Must not exist in POST requests, but required in all responses.
name	required	String <= 64 characters
description	required	String <= 2048 characters
category	optional	String <= 64 characters
price	required	Number >= 0
currency	optional	Currency code according to ISO 4217 (NOK, CZK, EUR, etc.), can be blank if price == 0 otherwise required

POST /api/articles

This request should accept a new article object and store it

Payload

JSON formatted article object

Response

JSON formatted article object with generated id

Typical example

```
1 POST /api/articles
2 {
3   "name": "Branded Memory Stick",
4   "description": "Branded 16 GB memory stick",
5   "category": "USB flash drive",
6   "price": 17.89,
7   "currency": "NOK"
8 }
```

Response:

```
1 HTTP 200 OK
2 {
3   "article_id": 1,
4   "name": "Branded Memory Stick",
5   "description": "Branded 16 GB memory stick",
6   "category": "USB flash drive",
7   "price": 17.89,
8   "currency": "NOK"
9 }
```

```
1 POST /api/articles
2 Payload:
3 {
4   "name": "Branded Drinking Mug",
5   "description": "Porcelain drinking cup with your logo on it.",
6   "category": "Mug",
7   "price": 0,
8 }
```

Response:

```
1 HTTP 200 OK
2 {
3   "article_id": 2,
4   "name": "Branded Drinking Mug",
5   "description": "Porcelain drinking cup with your logo on it.",
6   "category": "Mug",
7   "price": 0
8 }
```

Edge case:

Invalid input according to contract. Should return 400 Bad Request.

```
1 POST /api/articles
2 Payload:
3 {
4   "name": "Branded Memory Stick",
5   "description": "Branded 16 GB memory stick",
6   "category": "USB flash drive",
7   "price": 17.89,
8 }
```

Response:

```
1 HTTP 400 Bad Request
```

Invalid formatted number. Uses string instead of number.

```
1 POST /api/articles
2 Payload:
3 {
4   "name": "Branded Memory Stick",
5   "description": "Branded 16 GB memory stick",
6   "category": "USB flash drive",
7   "price": "17.89",
8   "currency": "NOK"
9 }
```

Response:

```
1 HTTP 400 Bad Request
```

GET /api/articles/{article-id}

This request will return the article object with the id specified in the url. Should respond with HTTP 404 Not Found if no article with that id exists.

Typical example:

```
1 GET /api/articles/1
```

Response:

```
1 HTTP 200 OK
2 {
3   "article_id": 1,
4   "name": "Branded Memory Stick",
5   "description": "Branded 16 GB memory stick",
6   "category": "USB flash drive",
7   "price": 17.89,
8   "currency": "NOK"
9 }
```

Edge case:

```
1 GET /api/articles/3
```

Response:

```
1 HTTP 404 Not found
```

GET /api/articles

Should return an array of the articles matching the criteria in the url. Possible criteria include name and category.

Request parameters

name: url-encoded string parameter. Should return articles with partial matches on name, not taking letter capitalization into account.

category: url-encoded string parameter. Should only return matches on the complete category name.

Typical example:

```
1 GET /api/articles?category=USB%20flash%20drive
```

Response:

```
1 HTTP 200 OK
2 [
3   {
4     "article_id": 1,
5     "name": "Branded Memory Stick",
6     "description": "Branded 16 GB memory stick",
7     "category": "USB flash drive",
8     "price": 17.89,
9     "currency": "NOK"
10  }
11 ]
```

```
1 GET /api/articles?name=Branded
```



Response:

```
1 HTTP 200 OK
2 [
3   {
4     "article_id": 1,
5     "name": "Branded Memory Stick",
6     "description": "Branded 16 GB memory stick",
7     "category": "USB flash drive",
8     "price": 17.89,
9     "currency": "NOK"
10  },
11  {
12    "article_id": 2,
13    "name": "Branded Drinking Mug",
14    "description": "Porcelain drinking cup with your logo on it.",
15    "category": "Mug",
16    "price": 0
17  }
18 ]
```

Edge case:

No records matching criteria. Should return empty array.

```
1 GET /api/articles?name=256%20GB
```

Response:

```
1 HTTP 200 OK
2 [
3 ]
```

Add unit tests

Implement a simple unit test (for example regarding the validations).

You can then add more unit tests for your solution but keep them in a reasonable count.

Extra tasks

Solve one or more of the following tasks. Only complete as much as the time allows for.

Concurrency

Add an endpoint that concurrently inserts a number of articles.

Example:

```
1 POST /api/articles-concurrent
2 Payload:
3 [
4   {
5     "name": "Branded Memory Stick",
6     "description": "Branded 16 GB memory stick",
7     "category": "USB flash drive",
8     "price": 17.89,
9     "currency": "NOK"
10  },
11  {
12    "name": "Branded Drinking Mug",
13    "description": "Porcelain drinking cup with your logo on it.",
14    "category": "Mug",
15    "price": 0
16  }
17 ]
```

Response:

```
1 HTTP 200 OK
2 [
3   {
4     "article_id": 1,
5     "name": "Branded Memory Stick",
6     "description": "Branded 16 GB memory stick",
7     "category": "USB flash drive",
8     "price": 17.89,
9     "currency": "NOK"
10  },
11  {
12    "article_id": 2,
13    "name": "Branded Drinking Mug",
14    "description": "Porcelain drinking cup with your logo on it.",
15    "category": "Mug",
16    "price": 0
17  }
18 ]
```

Edge case:

If validation of any article fails, return error:

```
1 HTTP 400 Bad Request
```



Documentation

The API should be well documented, add a swagger and some basic documentation for each endpoint.

Add integration tests

Add integration tests that will prove the whole application works correctly.

SQL Database

Implement the possibility for persistent storage in a database.

Update endpoint

Implement an endpoint for updating an article. Create a method that checks that the article hasn't been updated by a different user in the time since the article was loaded. You may add an attribute to the article.

Example:

```
1 PUT /api/articles/1
2 Payload:
3 {
4   "name": "Branded Memory Stick",
5   "description": "Branded 256 GB memory stick",
6   "category": "USB flash drive",
7   "price": "31.89",
8   "currency": "NOK"
9 }
```

Response:

```
1 HTTP 200 OK
```

Edge case:

If the article has been updated by another user in the meantime, return

1 HTTP 409 Conflict